

Quantization, LoRA & QLoRA

A Deep-Dive Class on Efficient LLM Fine-Tuning

Part 1: Model Quantization

Part 2: LoRA & QLoRA

Agenda

PART 1 — QUANTIZATION

1. What is Quantization? Definition & Motivation
2. Full Precision (FP32) vs Half Precision (FP16)
3. How Numbers are Stored in Memory
4. Symmetric vs Asymmetric Quantization
5. Scale Factor & Zero Point Calculations
6. Calibration Process
7. Post-Training Quantization (PTQ) vs Quantization-Aware Training (QAT)

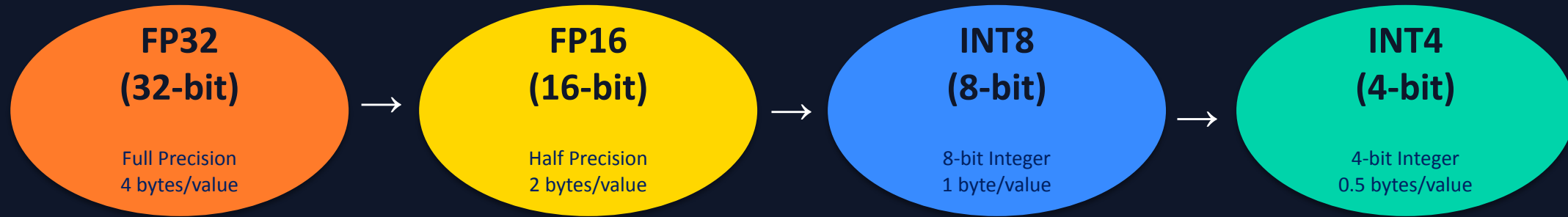
PART 2 — LoRA & QLoRA

1. Why Full-Parameter Fine-Tuning Has Limitations
2. Types of Fine-Tuning (Full, Domain-Specific, Task-Specific)
3. LoRA: Low-Rank Adaptation Explained
4. Matrix Decomposition Intuition
5. Rank Selection & Trainable Parameters
6. QLoRA: Quantized LoRA (LoRA 2.0)
7. Hands-On Code Example (Gemma + 4-bit)

What is Quantization?

Definition:

Quantization = Converting model weights from a HIGHER memory format to a LOWER memory format



Why does it matter?

LLMs have billions of parameters (weights) — e.g., LLaMA-2 70B has 70 billion

Storing them in FP32 requires enormous RAM/VRAM

Lower-bit storage = smaller model = faster inference + less memory

Enables deployment on edge devices, mobile phones & limited GPUs

How Numbers Are Stored in Memory

FP32 — Single Precision (32 bits)



Example: 7.23 → Sign: + | Exponent stores '7' | Mantissa stores '.23'

FP16 — Half Precision (16 bits)



Memory Comparison

Format	Bits	Bytes/Weight	Memory for 70B Model
FP32	32	4 bytes	~280 GB
FP16	16	2 bytes	~140 GB
INT8	8	1 byte	~70 GB
INT4	4	0.5 bytes	~35 GB

Symmetric vs Asymmetric Quantization

PART 1 — QUANTIZATION

Symmetric (Unsigned INT8)

Original range: 0 to 1,000

Quantized range: 0 to 255

$$\text{Scale} = (x_{\text{max}} - x_{\text{min}}) / (Q_{\text{max}} - Q_{\text{min}})$$

$$\text{Scale} = 3.92$$

Zero Point = 0

Example: $250 \div 3.92 \rightarrow \approx 64$

*Data evenly distributed around zero.
Zero point = 0.*

Asymmetric (Unsigned INT8)

Original range: -20 to 1,000

Quantized range: 0 to 255

$$\text{Scale} = (x_{\text{max}} - x_{\text{min}}) / (Q_{\text{max}} - Q_{\text{min}})$$

$$\text{Scale} = 4.0$$

Zero Point = 5

Example: $-20 \div 4.0 = -5 \rightarrow -5 + 5 \rightarrow 0$

*Data is skewed (not symmetric).
Zero point $\neq 0$ — shift is required.*

Calibration = The squeezing process — converting values from a high-precision format to a low-precision format using Scale & Zero-Point

High-Precision
Weights (FP32)

Calculate
Scale Factor

Calculate
Zero Point

Apply Formula:
 $\text{round}(x / \text{scale} + \text{zp})$

Quantized
Weights (INT8)

Two Key Parameters in Quantization:

Scale Factor

$$(x_{\text{max}} - x_{\text{min}}) / (Q_{\text{max}} - Q_{\text{min}})$$

Controls the compression ratio from original to quantized range

Zero Point

$$\text{round}(Q_{\text{min}} - x_{\text{min}} / \text{scale})$$

Handles asymmetric distributions — shifts data so minimum maps to 0

✓ In symmetric quantization: $\text{zero_point} = 0$ | In asymmetric: $\text{zero_point} \neq 0$

Modes of Quantization: PTQ vs QAT

PART 1 — QUANTIZATION

Post-Training Quantization (PTQ)

Process:

- 1 Pre-trained Model (fixed weights)
- 2 Apply Calibration (squeeze FP32→INT8)
- 3 Quantized Model Ready

Advantages:

- ✓ Simple & fast — no retraining needed
- ✓ Works on any existing pre-trained model

Disadvantages:

- ✗ Loss of accuracy due to precision loss
- ✗ Not used in fine-tuning workflows

Quantization-Aware Training (QAT)

Process:

- 1 Pre-trained Model
- 2 Apply Quantization
- 3 Fine-Tune with New Training Data
- 4 Quantized Fine-Tuned Model

Advantages:

- ✓ Recovers accuracy lost during quantization
- ✓ Standard approach for LLM fine-tuning (LoRA/QLoRA)

Disadvantages:

- ✗ Requires new training data
- ✗ Computationally heavier than PTQ

Quantization Summary

- ✓ Quantization converts FP32 weights → lower-bit formats (FP16, INT8, INT4)
- ✓ Scale Factor & Zero Point are the two key parameters
- ✓ Symmetric: zero_point = 0 | Asymmetric: zero_point ≠ 0
- ✓ PTQ = fast but loses accuracy
- ✓ QAT = used in fine-tuning (recovers accuracy with new data)
- ✓ Quantization enables LLM inferencing on limited hardware

Types of Fine-Tuning & Their Challenges

PART 2 — LoRA & QLoRA

Full Parameter Fine-Tuning

Update ALL model weights with new data

e.g., ChatGPT, Claude

Domain-Specific Fine-Tuning

Tune on a specific domain (Finance, Retail, Sales)

e.g., FinanceGPT, SalesBot

Task-Specific Fine-Tuning

Tune for a specific task (Text-to-SQL, Q&A, etc.)

e.g., SQL assistant, Doc QA

The Core Challenge with Full-Parameter Fine-Tuning

175B+ parameters need updating

For models like GPT-3, updating every weight requires massive compute

Extreme GPU/RAM requirements

Multi-GPU clusters needed — impractical for most companies/researchers

Costly for downstream tasks

Model monitoring, inference, serving all become expensive

Not scalable per use case

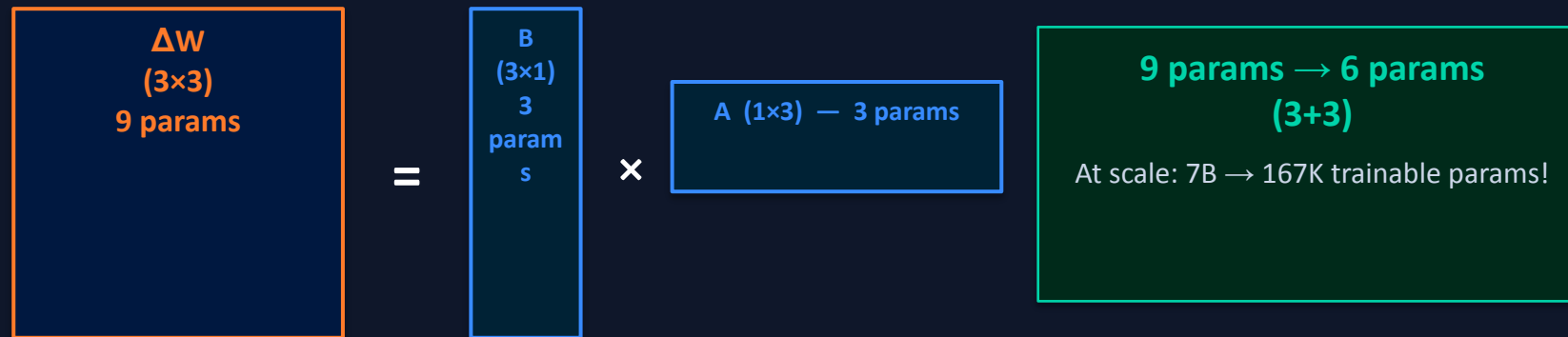
Can't afford to fully retrain a 70B model for every new task

LoRA: Low-Rank Adaptation of Large Language Models

PART 2 — LoRA & QLoRA

Core Idea: Instead of updating ALL weights during fine-tuning, LoRA tracks the CHANGES in weights and stores them in two smaller matrices via Matrix Decomposition

Matrix Decomposition: The Key Insight



LoRA Equation: $W_{\text{new}} = W_{\text{pretrained}} + B \times A$ (where W is frozen, only B and A are trained)

Trainable Parameters vs Model Size (Rank = 1):

7B model **167K**

13B model **228K**

70B model **529K**

180B model **849K**

vs 175B original params — all training options are a fraction of full fine-tuning!

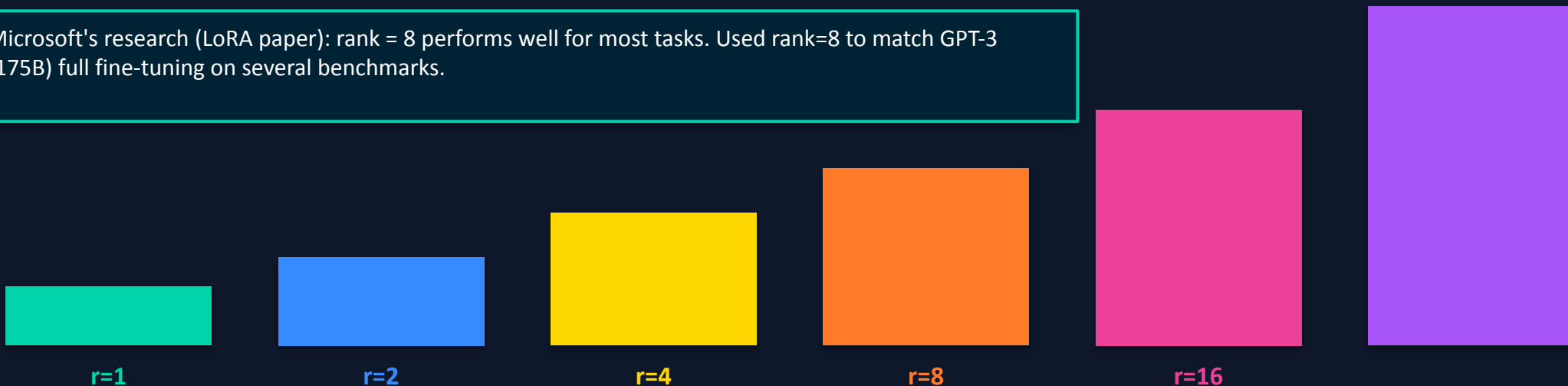
LoRA: Understanding Rank

What is Rank?

Rank controls the size of the decomposed matrices B and A. A higher rank $\rightarrow$ more trainable parameters $\rightarrow$ model can learn more complex patterns.

Trainable parameters grow with rank (but stay far below full fine-tuning):

Microsoft's research (LoRA paper): rank = 8 performs well for most tasks. Used rank=8 to match GPT-3 (175B) full fine-tuning on several benchmarks.



Use LOW rank (1–4)

Simple tasks, limited compute, domain adaptation

Use MEDIUM rank (8–16)

Most fine-tuning tasks — Microsoft's default recommendation

Use HIGH rank (32–64)

Complex behaviors model has never learned; very specialized tasks

QLoRA: Quantized LoRA (LoRA 2.0)

PART 2 — LoRA & QLoRA

QLoRA = Quantization + LoRA



Code Example (Gemma Model + 4-bit QLoRA):

```
# Quantization Config (bitsandbytes)
bnb_config = BitsAndBytesConfig(
    load_in_4bit = True,          # FP16 → 4-bit
    bnb_4bit_quant_type = 'nf4',  # NormalFloat4 format
    bnb_4bit_compute_dtype = torch.bfloat16 )

# LoRA Configuration
lora_config = LoraConfig(
    r = 8,                        # Rank
    target_modules = ['q_proj', 'v_proj'] # QKV attention
    task_type = 'CAUSAL_LM' )
```

Why QLoRA?

4× less memory

vs standard LoRA with FP16

No accuracy loss

NF4 format preserves quality

Reversible

Can dequantize back to FP16

Industry standard

Used in most OSS fine-tuning pipelines

Comparison: Full Fine-Tuning vs LoRA vs QLoRA

Metric	Full Fine-Tuning	LoRA	QLoRA
Trainable Params	175B (100%)	~4.7M (0.003%)	~4.7M (0.003%)
Memory Required	~700 GB (A100 cluster)	~80 GB (1–2 GPU)	~24 GB (1 GPU)
Training Data Needed	Huge dataset	Moderate dataset	Moderate dataset
Accuracy	Highest	Very close to full	Close to LoRA
Cost	Very High	Low	Very Low
Use Case	GPT-4, large orgs	Most fine-tuning	Limited GPU scenarios
Quantization Used	None	Optional	4-bit (NF4) required

Key Takeaway: QLoRA achieves near-full fine-tuning quality with a fraction of the compute. It is the standard technique used in most open-source LLM fine-tuning today.

Interview Prep: Key Questions & Answers

CHEAT SHEET

Q: What is quantization?

Converting model weights from FP32 to lower precision (FP16, INT8, INT4) to reduce memory and speed up inference.

Q: What is the difference between PTQ and QAT?

PTQ applies quantization after training (simple but loses accuracy). QAT fine-tunes the model with new data after quantization (recovers accuracy — used in LoRA/QLoRA).

Q: What is LoRA and why use it?

Low-Rank Adaptation. Instead of updating all weights, LoRA decomposes weight changes into two small matrices ($B \times A$). Reduces trainable params from 175B to ~4.7M — enabling fine-tuning on limited GPU.

Q: What is rank in LoRA?

Rank controls the size of decomposed matrices. Higher rank = more parameters = can learn more complex tasks. Microsoft uses rank=8 by default. Use high rank for very complex/novel behaviors.

Q: What is QLoRA?

Quantized LoRA — applies 4-bit quantization (NF4 format) to the base model BEFORE adding LoRA adapters. Reduces memory further (~24GB for a 7B model). Industry standard for LLM fine-tuning.

Q: What are Scale Factor & Zero Point?

Two parameters for quantization. Scale = compression ratio (x_range / Q_range). Zero Point = offset for asymmetric distributions. Symmetric: $zp=0$. Asymmetric: $zp \neq 0$.

Summary

Quantization:	FP32 → INT8/INT4 via Scale & Zero Point
Calibration:	The squeezing process — PTQ vs QAT
LoRA:	Matrix decomposition: 175B → ~5M trainable params
Rank Selection:	Higher rank = more complexity. Default: r=8
QLoRA:	4-bit quantized base + LoRA adapters = most efficient
Interview Ready:	Quantization, calibration, LoRA, rank, QLoRA — all covered!